Electronic & Computer Engineering
EEEN40280 Lab 1
2025/26

Accurate frequency synthesis on ADuC841 using Assembly Language & Interrupts

Thomas O'Dea (21386441)
Aidan Quigney (21388256)
Omotinuadewa Atilade (21328426)
Brightspace Group Number: 15

By submitting this report through Brightspace, we certify that ALL of the following are true:

1. We have read the Student Guide to the UCD Student Plagiarism Policy. (The full policy document is available on Brightspace). We understand the definition of plagiarism and the consequences of plagiarism.

2. We recognise that any work that has been plagiarised (in whole or in part) may be subject to penalties, as outlined in the documents above. That includes work that is the result of collaboration, where that is not acknowledged.

3. We have not plagiarised any part of this report. The work described was done by the people named above, and this report is all our own original work, except where otherwise acknowledged in the report.

February 15, 2026

Contents

Introduction **3**

Task 1: Software Timing **3**
Program Design . . . 3
Simulation Results . . . 4
Measured Results . . . 5
Code . . . 6

Example Hardware Program **6**
Expected Frequency . . . 7
Simulation Result . . . 7
Measured Result . . . 7

Task 2: Hardware timing **8**
Program Design . . . 8
Requirements . . . 8
Design Decisions . . . 8
Code . . . 11
Results . . . 14

Conclusion **16**

List of Figures

1 Logic Analyser waveform from a Keil uVision simulation showing exactly 18 periods in 20 ms . . . 5
2 Oscilloscope measurements of the 900 Hz square wave synthesised with software in both time domain (left) and frequency domain (right). . . . 5
3 Measurement of the frequency of the output waveform on the logic analyser for the example program . . . 7
4 Oscilloscope measurements of the example program's output in both time domain (left) and frequency domain (right). . . . 8
5 Time-domain measurements of all 8 frequencies synthesised . . . 15
6 Frequency domain view of the minimum and maximum frequencies . . . 16

Introduction

The aim of this laboratory exercise was to design and implement a frequency synthesiser using an ADuC841 microcontroller programmed in Assembly. The lab was intended to build understanding of timing and frequency generation in embedded systems, starting with software-based methods and progressing to hardware-based solutions using timers and interrupts.

As an initial step, we used the example program `blink.asm` as a reference. This program demonstrated basic Assembly, input/output pin control and the use of long loops to create delays in software.

In the first part of the assignment, a square-wave was generated using software delays only, without the use of hardware timers or interrupts. The output frequency was controlled by calculating the instruction execution times and adjusting the delay loops accordingly.

In the second part of the assignment, a square wave was generated using hardware timers which generated interrupts at regular intervals. First an example program was studied and run, then our own program was written which accommodated adjustable frequencies and several user-focused features.

Relying on dedicated timer hardware rather than software delays has the benefit of allowing the CPU core to perform other tasks, as well as providing a more precise frequency, as software delays can be unpredictable for reasons discussed below.

Task 1: Software Timing

Program Design

A sample program `blink.asm` was provided for initial demonstration of using software delays to generate a frequency. The target frequency was much lower and used to drive an LED rather than a transducer, but the software is similar.

This program was modified to provide a frequency of 900 Hz using a nested 'for' loop to delay the CPU core. The precise number of clock cycles to generate a 900 Hz square wave was calculated and the delay function tweaked accordingly. Only two levels of 'for' loop nesting were necessary for this faster period, rather than three levels in the original `blink.asm`.

The main program loop has only three instructions:

- Call a 'delay' to stall the CPU for a time
- Complement (flip) the output bit labelled 'SOUND' which is tied to the transducer on the physical board
- Return to the start of the loop

The ADuC841 runs with a clock speed of 11.0592 MHz. To accomplish a 900 Hz wave, the precise duration of this loop had to be precisely 6144 clock cycles, as given by equation 1. Note the output bit must flip twice per square wave period.

$$11.0592 \times 10^6 \frac{\text{clock cycles}}{\text{s}} = 6144 \frac{\text{clock cycles}}{\text{bit complement}} \times (2 \times 900) \frac{\text{bit complements}}{\text{s}} \tag{1}$$

	Instruction	Clock Cycles	Occurences per loop
MAIN:			
LOOP:	CALL DELAY	3+delay	1
	CPL SOUND	2	1
	JMP LOOP	3	1
DELAY:	MOV R5, #16	2	1
DLY1:	MOV R7, #126	2	16
	DJNZ R7, $	3	$2016 = 16 \times 126$
	DJNZ R5, DLY1	3	16
	NOP	1	1
	NOP	1	1
	RET	4	1
Total		6144	

Table 1: Calculation of main loop duration in clock cycles. See the actual Assembly in listing 1.

A delay function was designed to generate this precise period, accounting for the overhead in each loop. A table calculating the precise duration of the loop is shown in Table 1. Two `NOP` operations were included to artificially increase the loop duration.

Limitations of this design: Generally, software-based designs are not to be considered reliable for precisely timed applications for several reasons:

- In an embedded program which responds to interrupts, the interrupts could cause unpredictable delay between software loops, causing frequency jitter
- Software development is more difficult and timing will need to be recalculated whenever a functional change is made
- Some instructions on some MCUs have nondeterministic latency so software timing cannot be precise
- Keeping the CPU constantly busy could be considered a waste of power

Simulation Results

The code was first simulated in Keil uVision to verify performance. The number of clock cycles per loop was calculated as follows:

- Enter debug simulation mode
- Place a breakpoint on the command "CPL SOUND"
- Run the program to this point
- Note the "states" count for the simulation, indicating how many clock cycles have run so far
- Run the program until the breakpoint arrives again
- Note the "state" counts again and subtract the previous count, giving the total clock cycles from one bit flip to the next

The period was also measured using the logic analyser which gave a result in simulation time rather than clock cycles. This is shown in figure 1. All measurements were consistent with expectations.

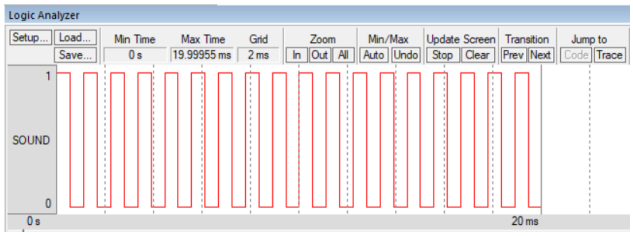


Figure 1: Logic Analyser waveform from a Keil uVision simulation showing exactly 18 periods in 20 ms

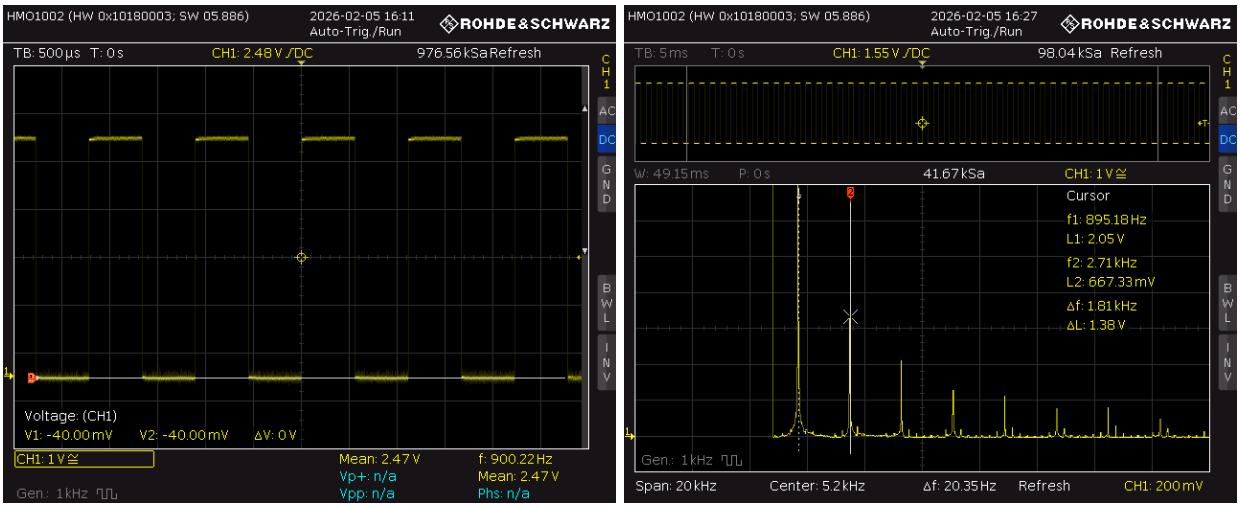


Figure 2: Oscilloscope measurements of the 900 Hz square wave synthesised with software in both time domain (left) and frequency domain (right).

Measurements were also taken using the 'stopwatch' functionality in the bottom-right-hand corner of uVision's interface. Results were identical.

Measured Results

The appropriate port on the ADuC841 was disconnected from the transducer and instead connected to an oscilloscope for measuring the square wave output. Figure 2 shows both a time-domain and frequency-domain measurements.

The oscilloscope measurement in time domain shows a frequency of 900.22 Hz, an error of only 244 parts-per-million. This error may arise from imperfections in the frequency of the 11.0592 MHz clock, or from the oscilloscope measurement itself.

The FFT view shows a fundamental frequency of 895.18 Hz. This is simply due to the granularity of the FFT jumping over the precise 900 Hz value. The expected series of harmonics for a square wave is present at odd multiples of the fundamental frequencies - the first such harmonic is highlighted with a cursor at a frequency of 2.71 kHz and an amplitude one-third of the fundamental's. Small spurs are present around each of the harmonics due to imperfections in the square wave.

Code

The full program is shown in listing 1.

```
;*******************************************************************
; 900 Hz Frequency Synthesiser
; Hardware      : ADuC841 with clock frequency 11.0592 MHz
;*******************************************************************

$NOMOD51            ; for Keil uVision - do not pre-define 8051 SFRs
$INCLUDE (MOD841)   ; load this definition file instead

SOUND       EQU P3.6        ; P3.6 is transducer on eval board
;-----------------------------------------------------------------
        ; MAIN PROGRAM
CSEG        ; working in code segment - program memory

        ORG 0000h           ; starting at address 0
MAIN:
LOOP:
        CALL  DELAY         ; call software delay routine       (3 cycles for jump + see subroutine)
        CPL   SOUND         ; flip the output bit 'SOUND'       (2 cycles)
        JMP   LOOP          ; return to loop start              (3 cycles)
        ; NOTE: next address is 0003h which is reserved for interrupts. If we need more setup instructions, we'll have to put in a jump to MAIN elsewhere
        ; Loop is 6144 clocks total giving precisely 900 Hz wave at 11.0592 MHz

;-----------------------------------------------------------------
        ; SUBROUTINES
DELAY:  ; Delay for using two nested FOR loops
        ; Delay is 6139 clocks including CALL and RET
        ; R5 and R7 are changed
        MOV R5, #16         ; set outer loop to 16 repetitions  (2 cycles)
DLY1:   MOV R7, #126        ; set inner loop to 126 repetitions (2 cycles * 16)
        DJNZ    R7, $       ; DJNZ inner loop                   (3 cycles * 126 * 16)
        DJNZ    R5, DLY1    ; DJNZ outer loop                   (3 cycles * 16)
        NOP                 ; (1 cycle)
        NOP                 ; Two "NOP" are necessary to get precisely 900 Hz (1 cycle each)
        RET                 ; return from subroutine (4 cycles)

;-----------------------------------------------------------------

END
```

Listing 1: Source code for generating 900 Hz wave purely in software

Example Hardware Program

An example program was provided to demonstrate the use of hardware timing. Full code is omitted for readability, but amounts to:

- An ISR is defined for overflows of timer 0
  - The ISR causes the output pin "SOUND" to complement

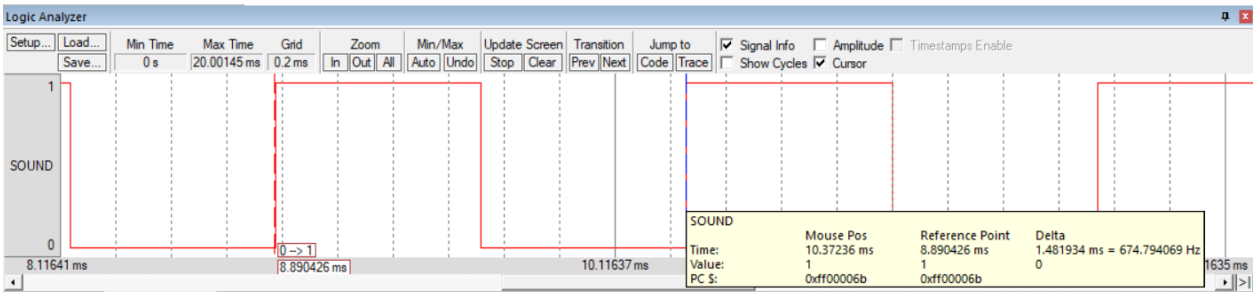


Figure 3: Measurement of the frequency of the output waveform on the logic analyser for the example program

- The main program sets timer 0 to ungated timer mode and enables the interrupt
- There is then an indefinite "NOP" loop which is regularly interrupted by the timer

This approach relies on a dedicated hardware timer to cause the interrupts, meaning unpredictable delays in software cannot cause the output waveform to jitter.

Expected Frequency

Timer 0 is configured as a 13-bit counter with no pre-scaler. This will overflow every $2^{13} = 8192$ clock cycles. At a frequency of 11.0592 MHz, this gives an interrupt every 740.74 microseconds. Each interrupt complements the output, so the period corresponds to two such interrupts, or 1.48148 milliseconds. This gives a frequency of precisely **675 Hz**.

Simulation Result

The program was simulated in Keil uVision and the period measured to be 674.794 Hz. A screenshot of the logic analyser is shown in figure 3.

Measurements were also taken in Keil using the stopwatch in the bottom-right-hand corner of the debug view. Strangely, the measured frequency varied slightly for each cycle, always between 674.9 and 675.1. Two causes for this variation may be:

- The uVision simulator has a bug or imprecision arising from floating-point errors
- The uVision simulator is modelling some non-idealities in the MCU itself

In any case, this variation is inconsequential.

Measured Result

The program was then executed on the physical board. Measurements on the oscilloscope are shown in figure 4.

The time-domain measurements shows a frequency of 675.12 Hz, which constitutes an error of 178 parts-per-million.

The frequency domain measurement shows the same harmonics as the previous exercise, again as expected for a square wave. The fundamental is measured at 667.57 Hz, again slightly off due to the granularity of the FFT measured frequencies.

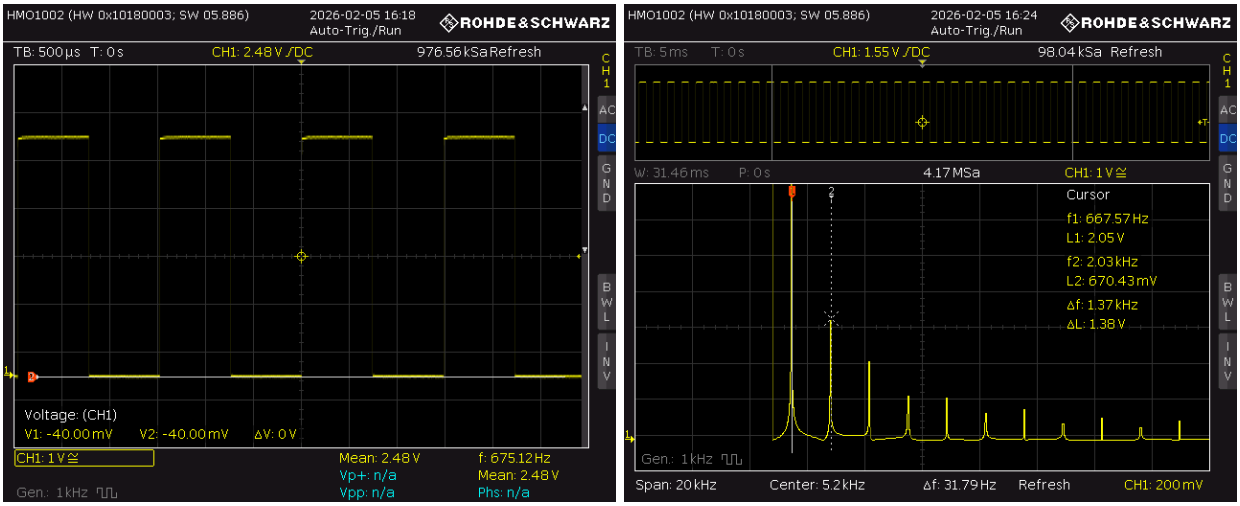


Figure 4: Oscilloscope measurements of the example program's output in both time domain (left) and frequency domain (right).

Task 2: Hardware timing

Program Design

Requirements

This task was the most substantial, having the following requirements:

1. Generate a square wave at an adjustable frequency using a hardware timer.
2. Allow the user to select the one of 8 frequencies using a set of 3 switches.
3. A panel of 8 LED's should indicate the selected frequency in one-hot code.
4. Another LED should flash at a rate visible to humans to give an indication the program is running (a "heartbeat"). Exact timing is not important.

Additionally, an optional feature was also implemented to have a pushbutton on the board toggle the heartbeat LED on an off. The physical button generates some bouncing when pressed, and no debouncing hardware is provide, so a software solution was developed.

Design Decisions

Choice of Frequency Synthesis Mechanism

In order to provide more precise frequency synthesis, this design relied on hardware interrupt rather than software delays or polling to generate the appropriate bit flips at the output.

Three philosophies for generating the adjustable frequency were proposed which variously used a timer generating interrupts with a fixed frequency, and software responding to those interrupts to control a divided frequency, or a timer which simply provides an adjustable frequency.

Timer 2 on the ADuC841 provides adjustable frequency via the autoreload mechanism. To investigate the viability of generating precise frequencies with this mechanism, the range and granularity of frequencies was studied. The frequencies which can be generated are given by:

$$f = \frac{11.0592 \text{ MHz}}{2 \times (2^{16} - n)} \quad \forall \quad n \in \{0, ..., 65535\} \tag{2}$$

This allows frequencies in the range 84.375 Hz to 5.5296 MHz, with significantly finer granularity at the lower end. However, only frequencies in the range 600 Hz to 9 kHz were of interest for this project. At the top end of this range, consecutive possible frequencies differ by 0.16%. Consecutive musical notes in the standard temperament differ by $2^{\frac{1}{12}} = 1+ \sim 5.9\%$, meaning this quantisation error should be inaudible to a human. Using overflow interrupts from timer 2 with an autoreload mechanism is therefore suitable for synthesising the frequencies of interest.

Choice of Frequencies

Frequencies of an E major triad were selected ranging from E5 (659.3 Hz) up to E8 (5.274 kHz). Appropriate reload values were calculated for each one by solving equation 2. These values were hardcoded into a lookup table in the software, allowing them to be reprogrammed easily.

It was considered to write an algorithm to calculate reload values based on Western musical frequencies being separated by a fixed factor of $2^{\frac{1}{12}}$. This would eliminate reliance on a hardcoded lookup table. However, it was decided that the software complications in this arithmetic were not justified.

Heartbeat LED Mechanism

While the timing for the heartbeat LED could have been implemented in software, it was preferred to avoid software delay functions and so another timer and interrupt was used. Timer 1 was configured in 16-bit timer mode to generate interrupts at the maximum period of $\frac{2^{16}}{1.0592\text{MHz}} =\sim$ 5.9 ms. When this interrupt was generated, it was used to increment register R1. When register R1 overflowed after 256 such interrupts, the LED would complement. This gave an LED flashing period of 3 seconds (1.5 seconds on, 1.5 seconds off). After physical testing this it was judged to be unnaturally slow, so the ISR was modified to increment R1 by 4, giving a period of 0.76 seconds.

Dedicating a second timer for the simple task of generating a flashing LED could be considered overkill if resources are constrained and the MCU has other tasks it also needs to accomplish. It is also possible to rely on the output of timer 2 and dividing with two registers appropriately to generate a sufficiently slow LED, with the consequence that the LED's blink rate will vary with the frequency being synthesised. Alternatively, if the time interval counter (TIC) is being used for other purposes, it is possible for the software to regularly poll the TIC and rely on sufficient time passing to blink the LED. For this project, timer 1 was used as there was no constraint not to do so.

Switchable Frequencies Mechanism

The panel of switches did not generate any interrupt when changed, so it was necessary to rely on software polling to respond to the switch positions. Assuming the main loop is fast enough to be imperceptible to humans, this is not an issue.

At each loop, the state of the 8 switches was read, and put through a bitwise AND with 0x07 to ignore all but the final three bits.

A lookup table was then used to determine the appropriate reload value for timer 2 to generate the appropriate frequency. The lookup table data was included into the program memory using the `DB` assembler directive, which allows the developer to explicitly include bytes program memory.

The data pointer `DPTR` would be pointed at the head of this lookup table, and the command `MOVC A,@A+DPTR` used to read the item at an *index equal* to the accumulator *into* the accumulator. This was repeated for the low byte and the high byte and (described later) the one-hot code for the display. A relevant code excerpt is shown in listing 2.

```
MOV A, SWITCHES      ; Load the switch info into A
ANL A, #07h          ; Mask away all but the last three bits
MOV DPTR, #RELOAD_VALUE_TABLE_L
; Prepare the data pointer with the table of lower bytes for the reload values
MOVC A, @A+DPTR      ; Read from the table into A
MOV RCAP2L, A        ; Write A to the timer 2 config register (lower byte)

RELOAD_VALUE_TABLE_L:
    DB  03Ch,   0FFh,   022h,   09Eh,   011h,   0CFh,   088h,   0E8h;
```

Listing 2: Minimal example of a lookup table in Assembly

A possible improvement to this main loop would be to detect changes in the switch state and skip the frequency lookup if there has been no change, which avoids an unnecessary write to the timer 2 autoreload register that could have unintended hardware consequence. In practice this does not seem to be an issue.

Display Panel Mechanism

A display panel of LED's had to show the frequency being generated. An active-low one-hot code had to be generated from the state of the switches. For example, switch 3 should generate the byte 0x11110111 to be written to the LED display.

Originally a bit-shifting mechanism was considered for generating the one-hot code, but this was found to require several clock cycles and a 'for' loop. Instead, a simple lookup table was used, which is looked up in the main software loop at the same time as the frequency lookup occurs.

Heartbeat LED Disable Button

A pushbutton was expected to disable the heartbeat LED mechanism when pressed, or re-enable when pressed again. The physical button experienced bouncing, so debouncing was necessary.

Initially, debouncing was to be implemented by comparing the timestamp of an interrupt with the timestamp of the previous successful interrupt, and ignoring rapidly successive interrupts. However, no appropriate timestamp was available as the TIC was not in use in the MCU. If the TIC were configured for other purposes regardless, its use would be appropriate here to avoid relying on software delays.

Instead debouncing was accomplished in software using software delays with the following method:

- The initial button press (edge-triggered) would create an interrupt request
- The ISR would respond immediately, disabling the heartbeat
- The ISR would delay for ∼50 ms for any further interrupts to arrive
- Any further pushbutton interrupts which arrived in that time were cleared before exiting the ISR

The ADuC841 does not allow an ISR to interrupt itself with another interrupt request, so it is sufficient to clear the interrupt flag at the end of one ISR to ignore any more that took place.

The software delay was implemented using a similar 'for' loop to that in task 1.

To disable the heartbeat LED, timer 1 would be disabled, and the LED explicitly turned off immediately. Switching off immediately, rather than waiting for the next natural off, allows the user to observe immediately that the button press was recognised. Similarly, when switching the heartbeat LED on, the LED is switched on immediately, and R1 is reset to ensure it does not switch off to soon, giving the user assurance that the button press worked. These improvements could be removed if program memory was constrained.

Program Summary

The initial setup to accommodate the program described above had the following steps:

- Configure T2CON to put timer 2 in timer mode with autoreload (done as a single full-register transaction).
- Configure TCON and TMOD to put timer 1 in 16-bit timer mode and disable timer 0, as well as ensuring INT0 (the pushbutton interrupt) is an edge-sensitive interrupt.
- Configure IE and IP to recognise the appropriate interrupts.

Three interrupt sources were allowed:

- Timer 2 overflow, used for frequency synthesis
- Timer 1 overflow, used for the heartbeat LED
- External interrupt INT0, used for the pushbutton

Timer 2 overflow was given **priority** to ensure the frequency did not jitter if another interrupt was already taking place when timer 2 overflowed.

This setup stage then fell through to a main loop which iteratively:

- Read the state of the switches
- Looked up the appropriate reload values for that frequency (low byte and high byte) and wrote to RCAP2L and RCAP2H
- Looked up the one-hot code for the LED display and wrote to "DISPLAY" (P0)

Code

The full program written is shown in listing 3.

```
; Written for ADuC841 evaluation board, with UCD extras.
; Generates a square wave on P3.6
; Frequency is selected using three switches
; A panel of 8 LED's displays the selected frequency
; Another LED flashes at slow intervals
; A pushbutton (INT0) can disable this flashing LED

; Include the ADuC841 SFR definitions
$NOMOD51
$INCLUDE (MOD841)

SOUND       EQU     P3.6        ; P3.6 will drive a transducer
PULSE       EQU     P3.4        ; P3.4 will drive a "heartbeat" or "pulse" LED
SWITCHES    EQU     P2          ; P2 are the eight input switches
```

```
DISPLAY     EQU     P0          ; there are LED's for displaying info to the user on P0

CSEG
;-----------------------------------------------------------------
; Define ISR jumps
;-----------------------------------------------------------------
        ORG     0000h       ; set origin at start of code segment
        JMP     MAIN        ; jump to start of main program

        ORG     0003h       ; External interrupt 0
        JMP     ISR_USER_BUTTON

        ORG     001Bh       ; Timer 1 overflow interrupt address
        JMP     ISR_TIMER_1

        ORG     002Bh       ; Timer 2 overflow interrupt address
        JMP     ISR_TIMER_2

        ORG     0060h       ; set origin above interrupt addresses

;-----------------------------------------------------------------
; Setup and main loop
;-----------------------------------------------------------------
MAIN:
        MOV     T2CON, #04h ; Configure Timer 2 (flags off, not using serial ports, ignore external input, run in timer mode with autoreload)
        MOV     TCON,  #01000001b   ; Configure timer 1 (simple 16-bit timer) and timer 0 (disabled) and INT0 as an external posedge-triggered interrupt
        MOV     TMOD,  #00010000b   ; Configure timer 1 to mode 1 (16-bit timer) and disable timer 0
        MOV     IE,    #10101001b   ; Disable interrupts except Timer 1, Timer 2, and external interrupt 0
        MOV     IP,    #00100000b   ; Give timer 2 interupt priority
        ; Fall through to main LOOP (indefinitely loop)
LOOP:
        MOV A, SWITCHES                     ; Load the switch info into R3
        ANL A, #07h                         ; Mask all but the last three bits
        MOV R3, A                           ; Store for future use

        MOV DPTR, #RELOAD_VALUE_TABLE_L     ; Prepare the data pointer with the table of lower bytes for the reload values
        MOVC A, @A+DPTR                     ; Read from the table into A
        MOV RCAP2L, A                       ; Write A to the timer 2 config register (lower byte)

        MOV A, R3                           ; Retrieve which index in the lookup table we want
        MOV DPTR, #RELOAD_VALUE_TABLE_H     ; Prepare the data pointer with the table of upper bytes for the reload values
        MOVC A, @A+DPTR                     ; Read from the table into A
        MOV RCAP2H, A                       ; Write A to the timer 2 config register (upper byte)

        MOV A, R3                           ; Retrieve which index in the lookup table we want
        MOV DPTR, #ONE_HOT_DECODE_BYTE      ; Prepare the data pointer with the table of one-hot coded bytes (0-7)
        MOVC A, @A+DPTR                     ; Read from the table into A
```

```
        MOV DISPLAY, A                      ; Write A to the display LED's to show which frequency is selected

        JMP     LOOP                        ; repeat main loop (checking switches)

;-----------------------------------------------------------------
; Lookup table for the reload value for timer 2 to give various frequencies
;-----------------------------------------------------------------
RELOAD_VALUE_TABLE_H:   DB  0DFh,   0E5h,   0EAh,   0EFh,   0F5h,   0F7h,   0FAh, 0FBh;
RELOAD_VALUE_TABLE_L:   DB  03Ch,   0FFh,   022h,   09Eh,   011h,   0CFh,   088h, 0E8h;
ONE_HOT_DECODE_BYTE:    DB  0FEh,   0FDh,   0FBh,   0F7h,   0EFh,   0DFh,   0BFh, 07Fh;
;                           E5      G#5     B5      E6      B6      E7      B7      E8
; Values are calculated such that (11.0592 MHz) / (2*(65536 - RELOAD_VALUE)) is the target frequency
; Output wave is square so expect to see harmonics

;-----------------------------------------------------------------
; Interrupt service routines
;-----------------------------------------------------------------

ISR_USER_BUTTON: ; user pressing the button can disable the 'heartbeat' LED
        ; Could consider moving this to the main loop instead of using an interrupt to keep code cleaner
        ; Toggle timer 1 (the behaviour of this button)
        CPL     TR1         ;

        ; It would be best if the light immediately came on/off so the user knows the button worked
        MOV     C, TR1      ; Copy TR1 to C as an intermediate storage
        CPL     C           ; Rerverse carry to change LED state
        MOV     PULSE, C    ; Copy C to PULSE

        ; If the light is being switched back on, best if it doesn't immediately toggle off, so reset the counter to 0
        MOV     R1, #0      ; reset the counter that toggles the light (just in case it was at 255, we don't want the light to immediately switch off leaving the user wondering if they pressed the button properly)

        ; Before exiting the ISR, call a delay to avoid bounces (other interrupts can still happen)
        MOV     R5, #5      ; load 5 as a duration argument for "DELAY" to give a ~50 ms delay. Could be interrupted by timer 2 but this is fine
        CALL    DELAY       ; call a ~50 ms delay to avoid another ISR due to bouncing

        CLR     IE0         ; ignore any subsequent interrupts that took place while this one was being handled (caused by bounces)
                            ; note this only works because ADuC841 will not let an ISR interrupt another ISR of equal priority

        ; Return
        RETI                ; Return from the ISR

ISR_TIMER_1: ; increment a counter in R1 - every time that counter overflows, toggle the pulse LED
```

```
        CLR     TF1             ; Clear the overflow flag that caused the ISR
        XCH     A, R1           ; Copy R1 into accumulator (we will undo at the end of the ISR)
        ADD     A, #4           ; Increment the accumulator
        JNZ     END_ISR_TIMER_1 ; If A is not zero, exit the ISR. We only toggle the LED each time A overflow
        CPL     PULSE           ; Flip the 'pulse' LED IF the counter has gotten to 0 (so every 64 times this ISR is called)
END_ISR_TIMER_1: ; fall through if no jump took place
        XCH     A, R1       ; Return A and R1 to their original positions
        RETI                ; Return from ISR

ISR_TIMER_2: ; timer 2 is the main timer used for frequency synthesis, toggle the SOUND pin each time it overflows
        CLR     TF2         ; Clear the overflow flag that caused the ISR
        CPL     SOUND       ; Flip 'sound' bit on timer 2 overflow
        RETI                ; Return from ISR

;-------------------------------------------------------------------
; SUBROUTINES
;-------------------------------------------------------------------

DELAY:  ; delay for time R5 x ~10 ms. Adapted from blink.asm. R5, R6, R7 are changed
        MOV     R6, #144        ; middle loop repeats 144 times
DLY1:   MOV     R7, #255        ; inner loop repeats 255 times
        DJNZ    R7, $           ; inner loop 255 x 3 cycles = 765 cycles
        DJNZ    R6, DLY1        ; + 5 to reload, x 144 = 110880 cycles
        DJNZ    R5, DELAY       ; + 5 to reload = 110885 cycles = 10.0264 ms
        RET                     ; return from subroutine

END
```

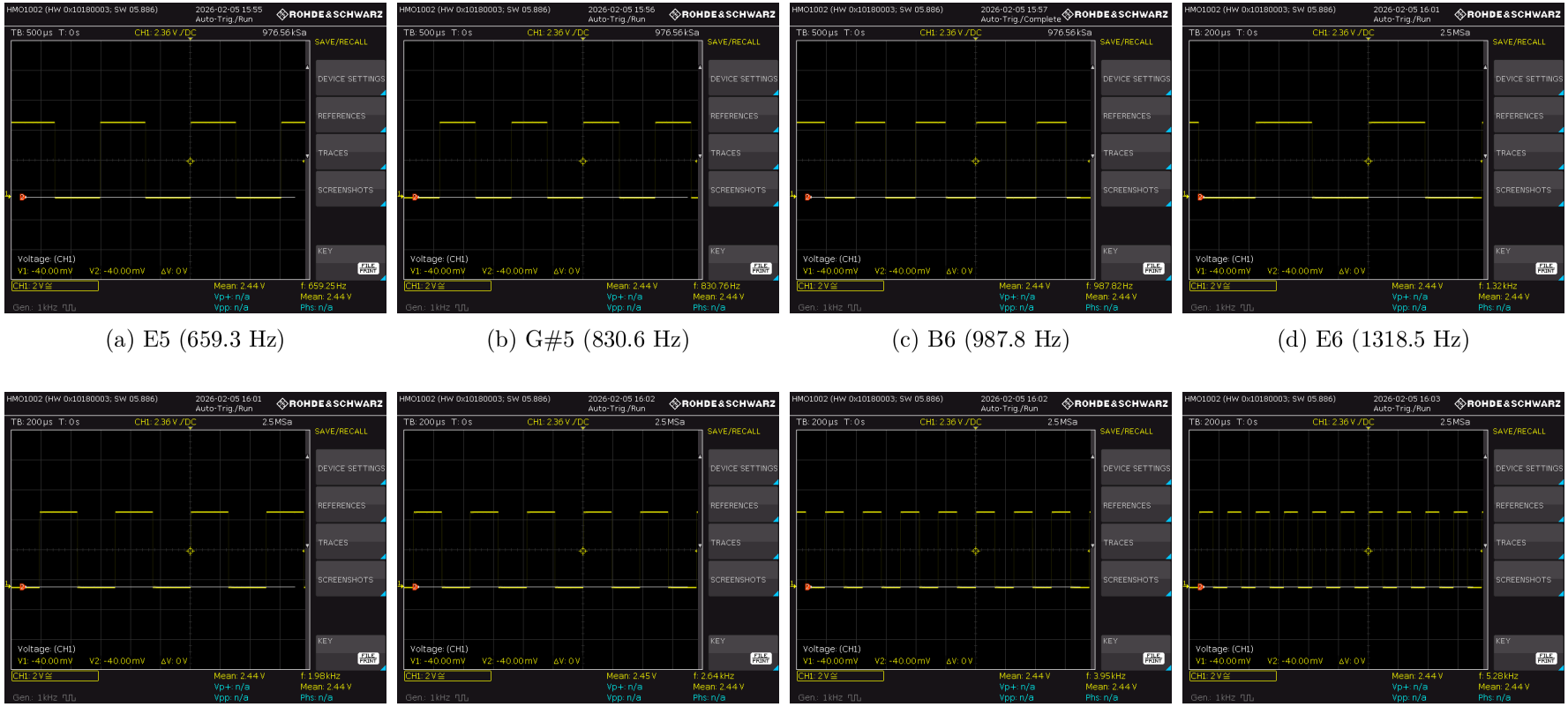
Listing 3: Assembly program to accomplish task 2. Users can select frequencies using a panel of switches on port 2. An LED display on port 0 shows the current selection. An LED pulses at regular intervals to show it is enabled. This LED can be disabled by a pushbutton.

Results

As before, the frequencies synthesised were measured on an oscilloscope in both the time and frequency domains. Figure 5 shows measurements of each frequency in the time domain, with the measured frequency visible in each one.

Figure 6 shows the FFT of the minimum frequency (E5, 692.3 Hz) and maximum frequency (E8, 5274.0 Hz).

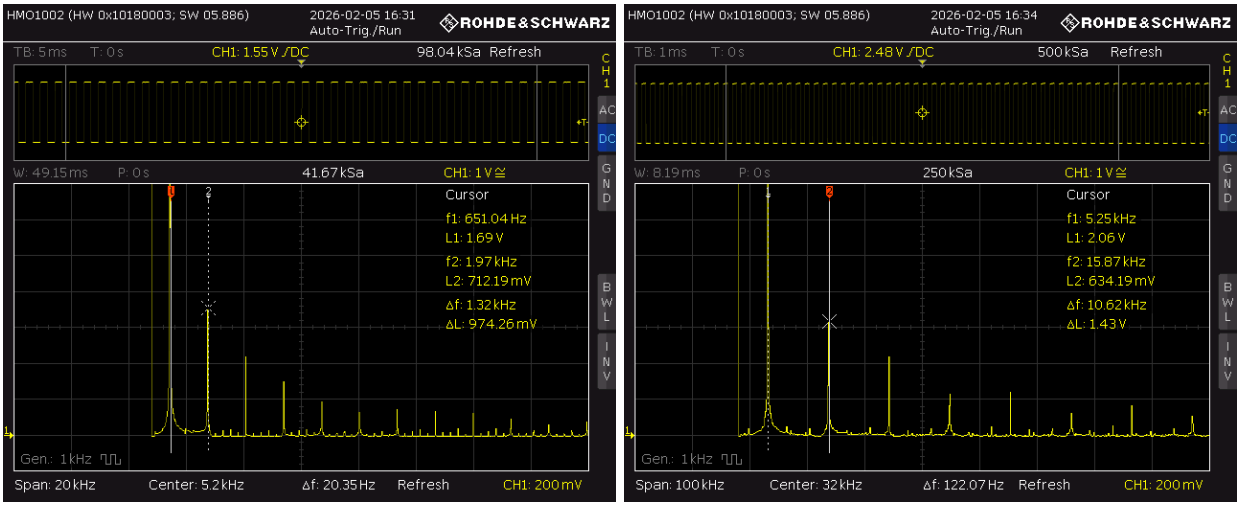
Musical coherence of the E major triad was confirmed aurally in the lab.



(a) E5 (659.3 Hz) (b) G#5 (830.6 Hz) (c) B6 (987.8 Hz) (d) E6 (1318.5 Hz)

(e) B7 (1975.5 Hz) (f) E7 (2637.0 Hz) (g) B8 (3951.0 Hz) (h) E8 (5274.0 Hz)

Figure 5: Time-domain measurements of all 8 frequencies synthesised



(a) E5 (659.3 Hz)

(b) E8 (5274.0 Hz)

Figure 6: Frequency domain view of the minimum and maximum frequencies

As expected there is a series of harmonics at odd multiples of the fundamental, which as before appears at the expected frequencies.

Conclusion

This lab has demonstrated the synthesis of frequencies using both pure software and mixed software/hardware approaches. An understanding of basic embedded programming in Assembly has been demonstrated, including the use of internal and external interrupts. Additional user-friendly features were included which required software-based debouncing of a physical pushbutton.

In all cases, the results expected from the design of the program are consistent with simulation results and with the measurements taken on the oscilloscope.